Universidad de San Carlos de Guatemala
Facultad de Ingeniería

Manual Técnico - Analizador Kanban

Nombre: Julio Eduardo Gustavo Alvarado Ramirez
Carnet: 202307764
Sección: A-
fecha: 27 de abril de 2026

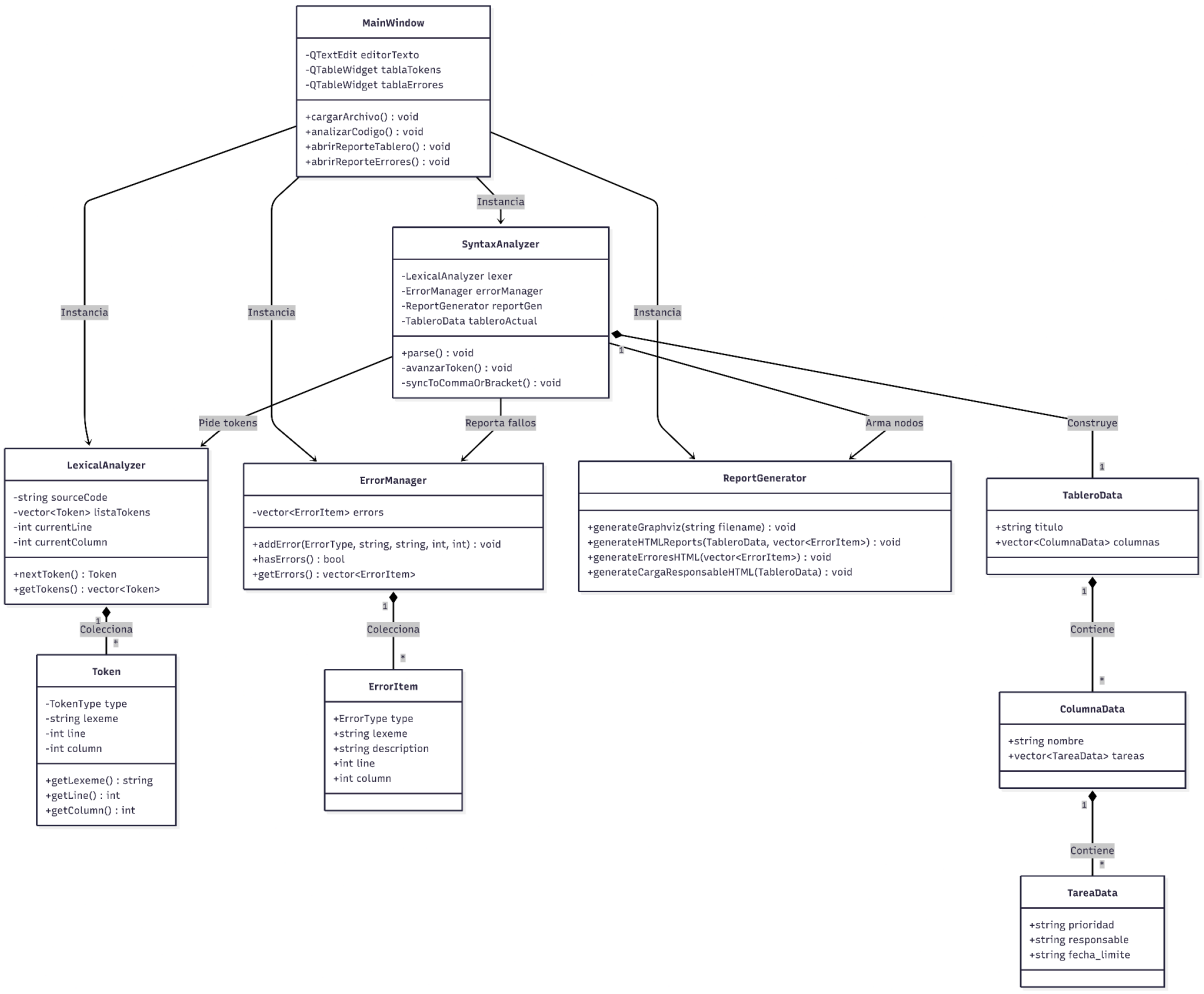
1. Arquitectura del Sistema

El proyecto utiliza una arquitectura de Pipeline (Tubería) clásica de compiladores, dividida en dos grandes bloques: el Front-End gráfico (Qt) y el Back-End analítico (C++ Nativo).

```
flowchart LR
    A[Archivo .task] -->|Carga de texto| B(LexicalAnalyzer)
    B -->|Flujo de Tokens| C(SyntaxAnalyzer)
    C -->|Reporta Fallos| D[ErrorManager]
    C -->|Construye Nodos| E[TableroData / Árbol]
    D --> F(ReportGenerator)
    E --> F
    F -->|Salida| G[Archivos HTML y Graphviz]
```

2. UML y Diseño Orientado a Objetos

El sistema aplica los principios de Single Responsibility (Responsabilidad Única) y Dependency Injection (Inyección de Dependencias). El SyntaxAnalyzer no crea sus propias instancias de gestión de errores o reportes, sino que las recibe por referencia desde el main o MainWindow, permitiendo que la interfaz gráfica consulte los resultados una vez terminado el análisis.



3. Diagrama del Autómata Finito Determinista (AFD)

El Analizador Léxico funciona como un AFD que lee el código fuente carácter por carácter. A continuación, se presenta la representación de los estados principales de reconocimiento:

(LL*|NNNN-NN-NN|NN*|"(L|N|w)*"|S)

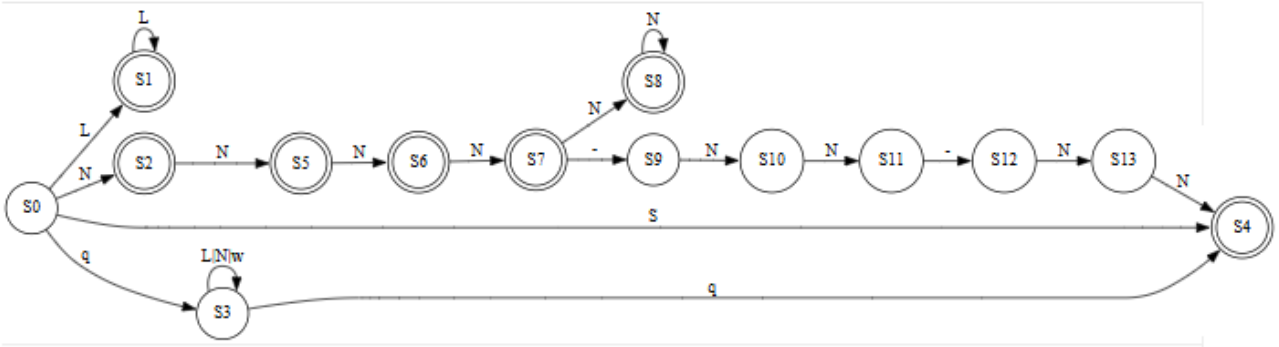
LL* = Identifica cualquier palabra (luego en código se verifica si es reservada como TABLERO, ALTA, etc.).

NNNN-NN-NN = Formato de fecha.

NN* = Números enteros.

"(L|N|w)*" = Cadenas de texto.

S = Delimitadores.



4. Gramática Libre de Contexto (GLC)

El analizador sintáctico se rige por la siguiente gramática LL(1), diseñada para eliminar la recursividad por la izquierda y permitir un análisis descendente predictivo:

No Terminales: <programa>, <columnas>, <columnas_prima>, <columna>, <tareas_opt>, <tareas>, <tareas_prima>, <tarea>, <atributos>, <atributos_prima>, <atributo>, <prioridad>

```
<programa>      ::= TABLERO CADENA "{" <columnas> "}"

<columnas>      ::= <columna> <columnas_prima>
<columnas_prima>::= <columna> <columnas_prima> | ε

<columna>       ::= COLUMNA CADENA "(" <tareas_opt> ")" ";"

<tareas_opt>    ::= <lista_tareas> | ε
<lista_tareas>  ::= <tarea> <tareas_prima>
<tareas_prima>  ::= "," <tarea> <tareas_prima> | ε

<tarea>         ::= tarea ":" CADENA "[" <atributos> "]"

<atributos>     ::= <atributo> <atributos_prima>
<atributos_prima>::= "," <atributo> <atributos_prima> | ε

<atributo>      ::= prioridad ":" <prioridad>
                  | responsable ":" CADENA
                  | fecha_limite ":" FECHA

<prioridad>     ::= ALTA | MEDIA | BAJA
```

Se modificó la gramática de referencia para adaptarla a un Parser Descendente Recursivo Predictivo (LL1). Se aplicó factorización izquierda y se eliminó la recursividad no determinista en las listas (columnas, tareas y atributos) introduciendo producciones 'prima' con transiciones vacías (épsilon). Además, se estandarizaron los delimitadores para respetar la jerarquía visual del lenguaje: llaves {} para tableros, paréntesis () para columnas y corchetes [] para los atributos de las tareas. Finalmente, se agregó la producción <tareas_opt> para soportar columnas vacías sin que el analizador genere un error sintáctico.

5. Descripción del Parser (Análisis Sintáctico)

Se implementó un **Parser Descendente Recursivo Predictivo** basado en un modelo LL(1) (Lee de izquierda a derecha, produce por la izquierda, mirando 1 token de adelanto).

Manejo y Recuperación de Errores (Modo Pánico)

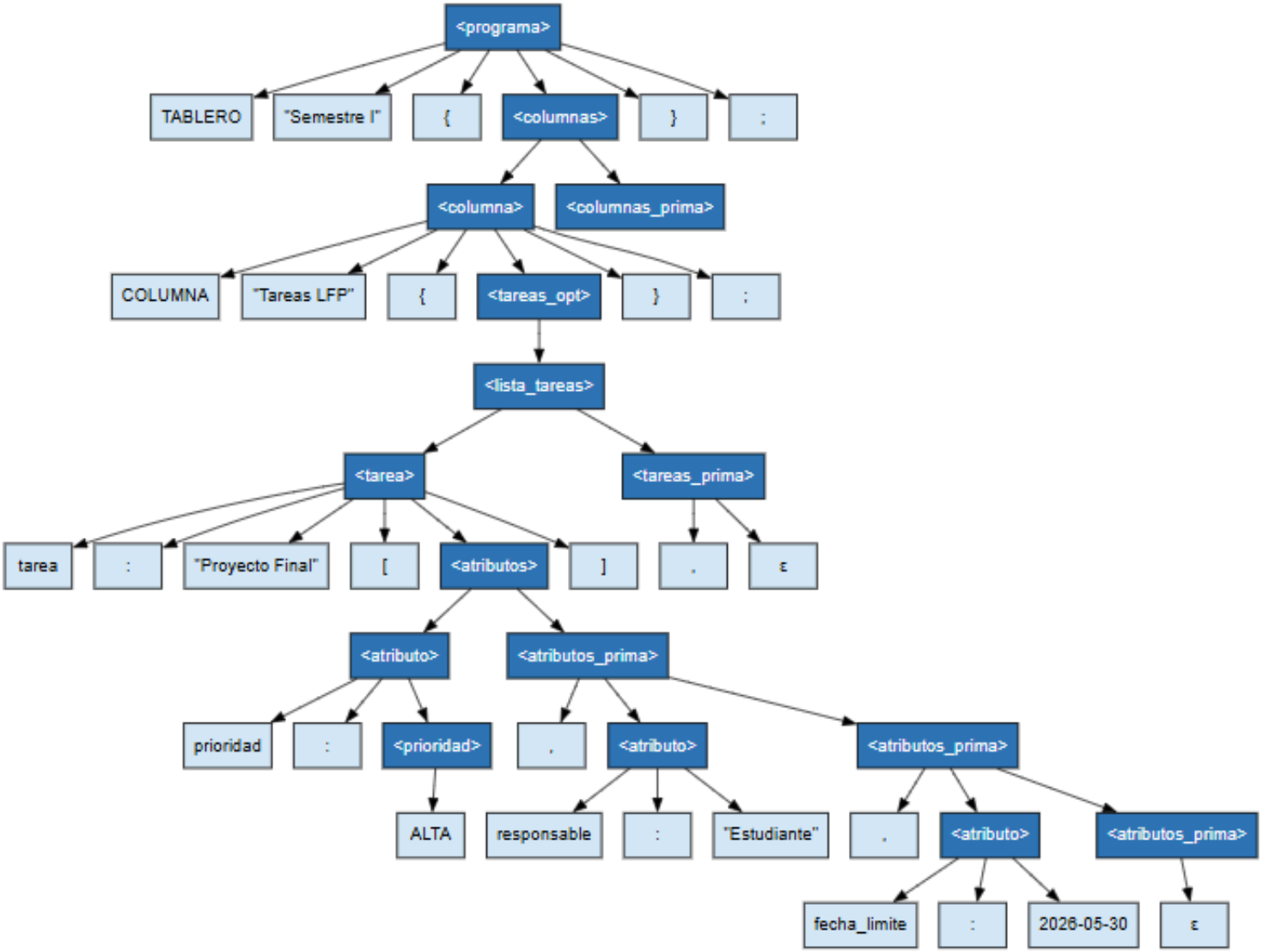
Para evitar que el compilador colapse ante el primer error (Efecto Cascada), se implementaron dos estrategias de recuperación:

1. **Filtro Antibasura Léxica:** El método avanzarToken() actúa como un middleware. Si el Lexer envía un error léxico, el filtro lo registra silenciosamente en el ErrorManager y pide el siguiente token, protegiendo al Parser de evaluar símbolos desconocidos.
2. **Sincronización Local:** Si se detecta un error de sintaxis profundo (por ejemplo, dentro de los atributos de una tarea), se activa un método de sincronización que consume tokens hasta encontrar un separador de rescate seguro (como una coma , o un corchete de cierre]). Esto aísla el error a una sola línea y permite que el análisis del resto del tablero continúe con normalidad.

6. Justificación de Decisiones Técnicas

Uso de C++ Moderno (C++17): Se eligió C++ por su control explícito de memoria y su rendimiento superior. El uso de estructuras del Standard Template Library (STL) como std::vector y std::map permitió gestionar las colecciones de tokens y calcular métricas de responsables con una complejidad temporal de O(N log N) y O(N) de forma segura y eficiente.

Interfaz en Qt6: En lugar de crear bindings hacia C# o Python, se utilizó Qt6 por ser nativo en C++. Esto elimina la necesidad de intermediarios (IPC o wrappers), reduciendo la latencia de análisis a 0. Además, su

sistema de Signals/Slots permite mantener la interfaz gráfica completamente desacoplada de la lógica del compilador.
Parser Manual (Descendente Recursivo) vs Generador (Bison/ANTLR): Se decidió programar el parser a mano porque proporciona un control granular absoluto sobre el árbol de llamadas. Esto fue crucial para poder inyectar rutinas de recuperación de errores personalizadas y reglas "indulgentes" como el trailing comma, las cuales son complejas de configurar en generadores automáticos estrictos.

6. Árbol de derivación